



## **Data Engineer Interview –**

**Duration- 1 Hour**

### **Scenario-Based Interview Simulation**

#### **Q1 — Project Introduction (5 min)**

##### **What they asked me:**

Explain your recent Data Engineering project end-to-end. Focus on data flow, tools used, and your role.

##### **What I said:**

In my recent project, I worked on building a customer analytics pipeline. The source data came from multiple systems — CRM database, application logs, and third-party APIs. Data arrived in both batch and streaming formats.

We ingested batch data using scheduled jobs and streaming data using event-based ingestion. Raw data first landed in the data lake storage in its original format. From there, we built transformation pipelines using Spark to clean, standardize, and enrich the data.

Transformations included:

- Deduplication using business keys
- Schema normalization

- Data quality validation checks
- Joining with master reference tables

After transformation, curated data was loaded into a warehouse layer optimized for reporting. We created partitioned tables and indexing strategies to support fast BI queries.

I was responsible for pipeline design, Spark job development, performance tuning, and production monitoring.

### Tips:

Always explain:

Architecture → Flow → Your contribution → Optimization → Business impact.

## Q2 — SQL Scenario (Joins + Duplicates) (6 min)

### What they asked me:

You have a customer table with duplicate records due to multiple ingestions. How would you remove duplicates but keep the latest record?

### What I said:

I would use a window function with ROW\_NUMBER ordered by last\_updated\_timestamp descending.

Example:

```
SELECT *
FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY last_updated_ts DESC
    ) rn
  FROM customers
) t
WHERE rn = 1;
```

This ensures only the most recent record per customer is retained. If performance matters on large tables, I would also ensure proper indexing or partition pruning.

### Tips:

Mention window functions — EXL interviewers expect strong SQL.

## Q3 — ETL Failure Scenario (5 min)

### What they asked me:

Your daily ETL job failed halfway after loading 40% data. How do you handle recovery?

**What I said:**

First, I check if the pipeline supports checkpointing or idempotent loads.

Best practice approach:

- Use batch IDs or load timestamps
- Track processed records
- Avoid blind append loads

Recovery steps:

1. Identify last successful checkpoint
2. Rollback partial load if required
3. Restart from last processed batch
4. Validate record counts after restart

In production systems, I design pipelines to be restartable so partial failures don't cause duplicate loads.

**Tips:**

Say "idempotent pipeline design" — strong keyword.

## **Q4 — Spark Performance Scenario (6 min)**

**What they asked me:**

Your Spark job is slow and taking 3 hours instead of 40 minutes. What do you check?

**What I said:**

I would analyze Spark UI first.

Key checks:

- Stage skew
- Shuffle size
- Executor memory usage
- Task distribution

Optimization actions:

- Repartition skewed keys
- Use broadcast joins for small tables
- Avoid wide transformations early
- Cache reused DataFrames

- Reduce shuffle by combining filters before joins

Example:

```
df = df.filter(col("status") == "active")
```

before join, not after.

### **Tips:**

Mention Spark UI — shows real-world experience.

## **Q5 — Data Skew Scenario (5 min)**

### **What they asked me:**

One key has 80% of records — Spark job is hanging. What do you do?

### **What I said:**

This is classic data skew. I would use salting technique.

Approach:

- Add random salt to skewed key
- Expand small table with salt keys
- Join using salted key

This distributes data across partitions and prevents one executor overload.

### **Tips:**

Say “salting technique” explicitly.

## **Q6 — Slowly Changing Dimension Scenario (6 min)**

### **What they asked me:**

Explain how you implement SCD Type 2.

### **What I said:**

SCD Type 2 preserves history by creating new records when attributes change.

Steps:

- Compare incoming record with existing
- If changed:

- Expire old record (set end\_date)
  - Insert new record with start\_date
- Use surrogate keys

Fields:

- effective\_start\_date
- effective\_end\_date
- is\_current\_flag

This helps track historical attribute changes.

### **Tips:**

Mention business use — audit + history tracking.

## **Q7 — Pipeline Design Scenario (6 min)**

### **What they asked me:**

Design a pipeline for ingesting API data every hour.

### **What I said:**

Design:

- Scheduler triggers ingestion
- API fetch with retry logic
- Store raw JSON
- Schema validation layer
- Transformation layer
- Curated warehouse layer

Error handling:

- Retry with backoff
- Dead-letter queue for failed records
- Logging + alerts

### **Tips:**

Always include retries + monitoring.

## **Q8 — Python Scenario (5 min)**

### **What they asked me:**

Memory issue when processing large CSV in Python?

**What I said:**

Avoid loading full file into memory.

Use chunk processing:

```
for chunk in pd.read_csv("file.csv", chunksize=100000):  
    process(chunk)
```

Or use Spark instead of pandas for large files.

**Tips:**

Chunking answer is expected.

**Q9 — Data Quality Scenario (5 min)****What they asked me:**

How do you ensure data quality in pipelines?

**What I said:**

I implement validation checks:

- Null checks
- Range checks
- Referential integrity
- Schema validation
- Duplicate detection

Failures go to quarantine tables. Pipeline doesn't silently pass bad data.

**Tips:**

Say "quarantine dataset".

**Q10 — Incremental Load Scenario (5 min)****What they asked me:**

How do you design incremental loads?

**What I said:**

Use:

- Watermark column (timestamp or ID)

- Store last processed value
- Filter new records only

Example:

```
WHERE updated_ts > last_run_ts
```

Also handle late arriving data using overlap windows.

### **Tips:**

Mention late-arriving data handling.

## **Q11 — Partition Strategy Scenario (4 min)**

### **What they asked me:**

How do you choose partition columns?

### **What I said:**

Choose columns:

- Frequently filtered
- High cardinality
- Even distribution

Example: date partition for time-series data.

Avoid:

- Low cardinality (like status)
- Highly skewed columns

### **Tips:**

Mention partition pruning.

## **Q12 — Real-Time Scenario (4 min)**

### **What they asked me:**

Batch vs Streaming — when to choose?

### **What I said:**

Batch:

- Historical data
- Cost efficient
- Non-urgent analytics

Streaming:

- Fraud detection
- Monitoring
- Real-time alerts

Decision based on latency requirements.

**Tips:**

Tie to business need.